Vehicle Damage System - Technical Spec (handoff for upgrade review)

1. Context & hard constraints

- **Stack:** three.js (WebGL) + Rapier `@dimforge/rapier3d-compat` (Rust->WASM rigid-body physics), vanilla JS, Vite, static deploy (GitHub Pages). No engine switch, no Rapier fork, no soft-body solver available.
- **Sim loop:** fixed timestep 60 Hz with render interpolation. All vehicle logic is custom JS (Rapier's built-in vehicle controller is NOT used).
- **Vehicle model:** chassis = one dynamic rigid body with a cuboid collider + explicit mass/inertia (`setAdditionalMassProperties`, low CoM). Wheels are raycast suspension (4 corner rays, Hooke spring + damper, per-wheel Pacejka-style lateral slip curve, friction circle, wheelspin state with kinetic-friction grip loss). **The chassis collider never touches the ground in normal driving** - it floats on the suspension.
- **Performance budget:** must eventually run ~20 vehicles simultaneously in-browser. Current damage costs are all event-gated (zero per-frame cost while driving). Debris is deliberately NOT physics bodies.
- **Car assets:** Synty low-poly FBX, loaded as ONE baked static body mesh (vertex-welded at load with `mergeVertices` so it deforms as a continuous skin) + 4 separate wheel meshes. No separate door/hood/bumper submeshes exist on these models.

2. Impact detection

- The chassis collider has Rapier `CONTACT_FORCE_EVENTS` enabled with a force threshold. Because the chassis floats on suspension, ANY chassis contact is a genuine impact (no rolling/resting-contact filtering needed).
- Per event we extract: **total force magnitude** (N, the severity scalar), **world contact point** (from the contact manifold; falls back to force direction if unavailable), **force direction**.
- Measured calibration: a rigid wall impact produces ~ **22,000 N per km/h of closing speed**.

3. Severity ladder (single scalar -> tiered consequences)

Force (N)	~ closing speed	Effect
250k	15 km/h	body dent (deform)
700k	32 km/h	glass shatters
1.1M	50 km/h	wheel tears off (if contact within 1.6 m of a wheel)
1.6M	72 km/h	debris chunks + deeper gouge
2.2M	100 km/h	auto slow-mo replay (cosmetic)

One impact can trigger multiple tiers. Camera shake and spark-count also scale from the same scalar.

4. Visual deformation (crumple) - the core algorithm

Runs once per qualifying impact on the welded body mesh (`BufferGeometry.position`):

```
deform(worldPoint, mag, worldDir):
  if mag < 250k: return
  sync visual to the impact pose; M = inverse(bodyMesh.matrixWorld); s = uniform world scale
```

```
epicenter = worldPoint transformed by M      // body-mesh local space
pushDir   = normalize(bodyCenter - epicenter)     // panel caves toward body center
sev    = clamp((mag - 250k) / 2M, 0, 1)
radius = (0.9/s) * (0.5 + sev)
depth  = (0.6/s) * sev
for each vertex v within radius of epicenter:
  fall = (1 - dist/radius)^2                      // deepest at epicenter
  v += pushDir * depth * fall
  clamp |v - pristine(v)| <= 0.5/s                // accumulation cap; can't collapse shell
positions.needsUpdate; recompute vertex normals   // dents catch light
```

- Pristine vertex array is stored at load (used for the clamp and for repair).
- Deform accepts an explicit pose so the replay system can re-enact dents at recorded moments.
- **Key limitation: this is visual-only. The physics collider (pristine cuboid) is never modified.**

5. Glass

Glass materials auto-detected at model load (material name contains "glass" or transparent+opacity<0.92). On threshold: glass material opacity->0 (visual removal) + 12-20 tetrahedron shards spawned around the greenhouse (positioned/rotated with the car pose), flung with car velocity + randomized pop. Restored on reset. Binary state (intact/shattered), whole-car (not per-window).

6. Wheels - the only *mechanical* damage

- Detach: nearest attached wheel within 1.6 m of the contact when force >= 1.1M. Wheel mesh reparented to the scene as debris (car velocity + outward pop + spin).
- Physics consequence (real handling change): that corner's suspension switches to a "bare hub" model - shorter rest length (sits lower), 2.5x damping (no pogo), **zero tyre forces**, plus a horizontal scrape force opposing motion (magnitude ~1.1x the corner load, capped). Result: top speed drops ~60%, the car pulls toward the dead corner, sparks emit from the scraping hub.
- Reset re-fits wheels (position/rotation/scale restored).

7. Debris (all loose pieces)

Chunk tier spawns 4-8 small tetra/box fragments (some body-coloured). All debris - wheels, chunks, glass - is integrated in plain JS: velocity, gravity, ground bounce (damped), spin decay, fade-out ~6 s, hard cap on live pieces. **No physics bodies** (intentional, for the 20-car budget). Debris does not interact with vehicles.

8. Integrations

- **Particles:** GPU point-sprite engine (custom shader, per-particle size/alpha/drag/gravity). Tyre smoke (heat-progressive), impact sparks, exhaust flames.
- **Audio:** procedural (Web Audio) - impact crash samples, slip-driven tyre squeal, suspension clunk one-shots, backfire pops.
- **Replay:** 60 Hz pose ring buffer (~8 s) + a damage-EVENT log `{frame, point, mag, dir, pose, glassFlag}`. Replay starts the car pristine and re-applies each event at its recorded moment using its recorded pose; fast-forwards remaining events on exit. Detached wheels/debris are not yet rewound.

9. Known limitations (the upgrade targets)

1. **Deformation is visual-only** - the collision shape stays a pristine box regardless of damage.

2. **Single skin, no zones** - no per-panel identity/health (hood vs door vs quarter); dents are purely local geometry.
3. **No structural deformation** - the chassis frame never bends/twists; suspension geometry, steering, and wheel mounts are unaffected by damage.
4. **No crush/pinch handling** - being sandwiched between objects is just two ordinary impacts.
5. **Mechanical damage is wheels-only** - no engine/radiator/steering/axle damage states.
6. Debris/glass don't persist into or rewind within replays; debris doesn't collide with vehicles.

10. What we want from this review

Target quality bar: **Wreckfest-style** damage (deep panel crumple, parts separation, mechanical degradation), NOT BeamNG soft-body. Constraints in §1 are firm (rigid-body Rapier, browser, ~20 cars, fixed 60 Hz).

Questions:

1. Best-practice approach for **collider feedback** - making accumulated deformation affect collision shape affordably (e.g., periodic convex-hull recompute from deformed verts? compound collider of a few movable cuboids? SDF-ish proxy?), given Rapier's collider set is static shapes.
2. Recommended architecture for **damage zones** on a single welded mesh (vertex-region tagging at load? spatial panels? bone/weight painting offline?) and per-zone health/consequences.
3. Cheap **structural deformation**: faking frame bend (e.g., persistent per-corner suspension mount offsets / steering misalignment driven by accumulated zone damage) vs. actually skewing the mesh - what reads best at Wreckfest fidelity?
4. **Crush detection** in a rigid-body world (opposing simultaneous contacts?) and an affordable response.
5. A sensible **mechanical damage model** (engine/cooling/steering states) driven by the existing severity scalar + zones.
6. Priority order for the above under a browser/20-car budget.